Chapter 1

L2 - SQL Introduction: Part 1

1.1 Relational terminology

Although we already defined the relational model, let's align our vocabulary with SQL:

- **Relation** translates to a **table**.
- **Tuple** translates to a **row**.
- **Attribute** translates to a **column**.

Throughout this chapter, we use the following example `Students` table:

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@ee	18	3.2
53777	White	white@cs	19	4.0

Primary Key (PK). The column (or group of columns) that uniquely identifies a row. Two rows **cannot** share the same PK. Above, `sid` is the primary key.

1.2 Basic SQL queries

1.2.1 The simplest query

To retrieve all contents of a table, use `SELECT *`:

```
SELECT * FROM Students S
```

`S` is an optional *alias* (range variable) for the `Students` table, useful when referencing columns. This returns the full table shown above.

1.2.2 Showing specific columns (Projection)

To retrieve only certain attributes, replace `*` with column names:

```
SELECT S.name, S.login FROM Students S
```

Projection. Selecting specific columns from a table. The query above projects `name` and `login` from `Students`.

name	login
Jones	jones@cs
Smith	smith@ee
White	white@cs

1.2.3 Showing specific rows (Selection)

To filter rows, use the `WHERE` clause:

```
SELECT * FROM Students S WHERE S.age = 18
```

Selection. Filtering rows based on a condition. The query above selects students whose age equals 18.

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@ee	18	3.2

1.2.4 SQL vocabulary

The general form of a basic SQL query:

```
SELECT [DISTINCT] target-list FROM relation-list WHERE qualification
```

- **relation-list**: a list of table names, each possibly followed by an alias.
- **qualification**: comparisons combined with `AND`, `OR`, `NOT`. Each comparison: `Attr` *op* `Const` or `Attr1` *op* `Attr2`.
- **target-list**: a list of attributes from the tables in *relation-list*.
- **DISTINCT**: optional keyword; eliminates duplicate rows. By default, SQL keeps duplicates (the result is a *multiset*).

1.2.5 Evaluation order

Conceptually, a SQL query is evaluated as follows:

1. **FROM**: compute the cross-product of all tables.
2. **WHERE**: discard tuples that fail the condition (*selection*).
3. **SELECT**: keep only the requested columns (*projection*).
4. If **DISTINCT** is specified, eliminate duplicate rows.

1.3 Querying multiple relations

To join two tables, list them both in `FROM` and express the join condition in `WHERE`. Consider the following `Enrolled` table alongside `Students`:

sid	cid	grade
53831	Carnatic101	C
53831	Reggae203	B
53650	Topology112	A
53666	History105	B

"Find all students with grade B":

```
SELECT S.name, E.cid
FROM Students S, Enrolled E
WHERE S.sid = E.sid AND E.grade = 'B'
```

S.name	E.cid
Jones	History105

1.3.1 Naïve evaluation in steps

The evaluation of the query above proceeds conceptually as follows:

Step 1 – Cross Product (FROM): Combine every row of `Students` with every row of `Enrolled`:

S.sid	S.name	S.login	S.age	S.gpa	E.sid	E.cid	E.grade
53666	Jones	jones@cs	18	3.4	53831	Carnatic101	C
53666	Jones	jones@cs	18	3.4	53831	Reggae203	B
53666	Jones	jones@cs	18	3.4	53650	Topology112	A
53666	Jones	jones@cs	18	3.4	53666	History105	B
53688	Smith	smith@ee	18	3.2	53831	Carnatic101	C
53688	Smith	smith@ee	18	3.2	53831	Reggae203	B
53688	Smith	smith@ee	18	3.2	53650	Topology112	A
53688	Smith	smith@ee	18	3.2	53666	History105	B

Step 2 – Apply the predicate (WHERE): Keep only rows where `S.sid = E.sid` **and** `E.grade = 'B'`. Only one row survives: Jones (sid 53666) enrolled in History105 with grade B.

Step 3 – Project the requested columns (SELECT): Keep only `S.name` and `E.cid`, yielding the final result: `(Jones, History105)`.

Running example: Sailors, Boats, Reserves

The following three relations are used throughout the rest of this chapter.

Sailors

sid	sname	rating	age
22	Dustin	7	45.0
31	Lubber	8	55.5
95	Bob	3	63.5

Boats

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Reserves

sid	bid	day
22	101	10/10/96
95	103	11/12/96

1.4 Aggregate operators

Aggregate function. A function that operates on a set (or multiset) of values from a column and returns a **single** summary value. Aggregates are a significant extension of relational algebra.

Function	Description
`COUNT(*)`	number of rows
`COUNT([DISTINCT] A)`	number of (distinct) non-null values in `A`
`SUM([DISTINCT] A)`	sum of (distinct) values
`AVG([DISTINCT] A)`	average of (distinct) values
`MAX(A)`	maximum value
`MIN(A)`	minimum value

1.4.1 Examples

Find the number of sailors:

```
SELECT COUNT(*) FROM Sailors S
```

Find the average age of sailors with rating 10:

```
SELECT AVG(S.age) FROM Sailors S WHERE S.rating = 10
```

Count distinct ratings of sailors named Bob:

```
SELECT COUNT(DISTINCT S.rating) FROM Sailors S WHERE S.sname = 'Bob'
```

1.4.2 Name and age of the oldest sailor(s)

This example illustrates a common pitfall with aggregates.

Incorrect query

```
SELECT S.sname, MAX(S.age) FROM Sailors S
```

This is **wrong**: `MAX` returns a single value over the entire column, so there is no meaningful way to pair it with a particular `sname`.

Correct approach — subquery

```
SELECT S.sname, S.age
FROM Sailors S
WHERE S.age = (SELECT MAX(S2.age) FROM Sailors S2)
```

The inner query computes the maximum age; the outer query retrieves every sailor matching that value.

Scalar subquery. A subquery that returns exactly one row and one column. It can appear wherever a single value is expected (e.g. in a `WHERE` comparison).

An equivalent alternative places the subquery on the left:

```
SELECT S.sname, S.age
FROM Sailors S
WHERE (SELECT MAX(S2.age) FROM Sailors S2) = S.age
```

1.5 GROUP BY

So far, aggregates have been applied to *all* qualifying tuples at once. Sometimes we need to apply them to each of several *groups* independently.

Consider: find the age of the youngest sailor for each rating level.

- We don't know in advance how many rating levels exist or what their values are.
- If ratings range from 1 to 10, we *could* write 10 separate queries:

For $i = 1, 2, \ldots, 10$:

```
SELECT MIN(S.age) FROM Sailors S WHERE S.rating = i
```

This is obviously impractical.

GROUP BY clause. Partitions tuples into groups based on equal values of the specified columns, then applies aggregate functions independently to each group.

1.5.1 Using GROUP BY

We use an extended `Sailors` table for the following examples:

sid	sname	rating	age
22	Dustin	7	45.0
31	Lubber	8	55.5
95	Bob	3	63.5
52	Smith	3	74.5
68	John	7	56.5
81	Ana	7	71.5

"Find the youngest age for each rating level":

```
SELECT S.rating, MIN(S.age) FROM Sailors S GROUP BY S.rating
```

The tuples are first partitioned by `rating` (groups: rating 3, rating 7, rating 8), then `MIN(S.age)` is evaluated within each group:

rating	MIN(age)
3	63.5
7	45.0
8	55.5

GROUP BY semantics. The `SELECT` clause may only contain:

- attributes listed in the `GROUP BY` clause, and
- aggregate functions applied to other attributes.

Any non-aggregated column in `SELECT` that is *not* in `GROUP BY` is an error.

1.5.2 HAVING — filtering groups

`HAVING` filters *groups* (after grouping), just as `WHERE` filters *individual tuples* (before grouping).

"Find the youngest age for each rating level that has **at least two** sailors":

```
SELECT S.rating, MIN(S.age)
FROM Sailors S
GROUP BY S.rating
HAVING COUNT(*) > 1
```

Groups with only one member (rating 8, containing only Lubber) are eliminated:

rating	MIN(age)
3	63.5
7	45.0

HAVING clause. A filter applied *after* grouping. It uses aggregate conditions to keep or discard entire groups, unlike `WHERE` which operates on individual rows before any grouping.